# ACT02. Interfaces y componentes

Repositorio: github.com/paunadal14-web/HabitosAPP
Usuario: pauchava.

## 1. Organización y navegación

| Parte | Responsabilidad |
|---|---|
| main.dart | Iniciar la aplicación, cargar los ajustes y configurar el tema. |
| pantallas/ | Mantener el estado y los controladores, navegar y llamar a los servicios. |
| widgets/ | Mostrar los datos recibidos y ejecutar las acciones mediante callbacks. |
| modelos/ | Representar usuarios, hábitos y estadísticas. |
| servicios/ | Acceder a SQLite y a las preferencias, y comprobar los permisos. |

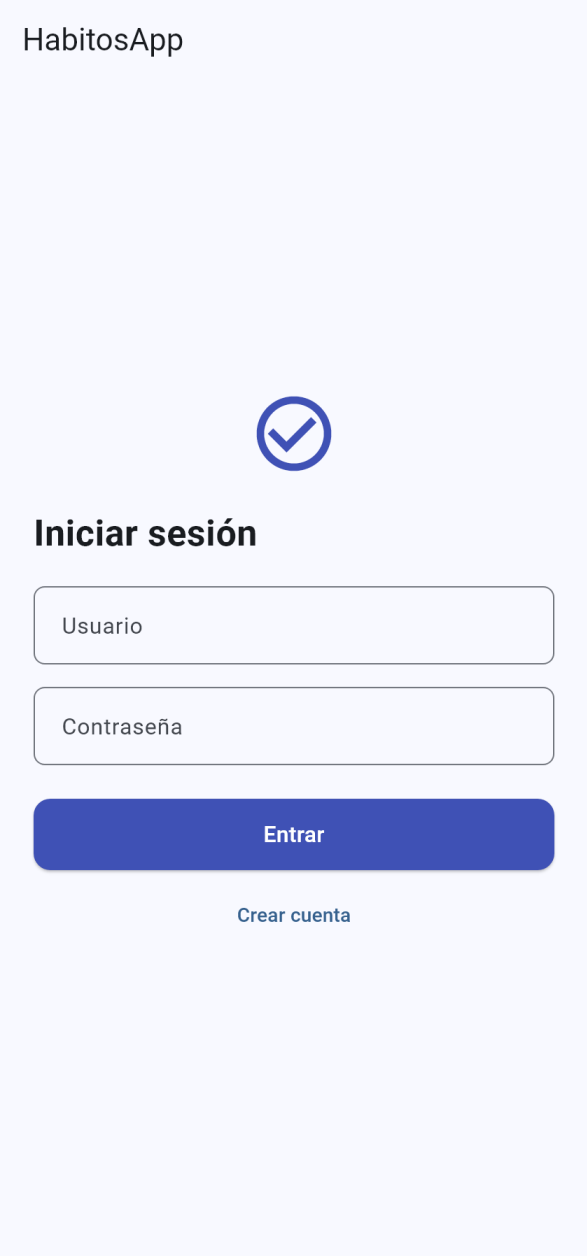
### Recorrido principal

```
Inicio de sesión -> Registro -> Mis hábitos
Inicio de sesión -> Mis hábitos -> Crear o editar hábito
Mis hábitos -> Estadísticas / Ajustes / Cerrar sesión
Mis hábitos -> Administración (solo administrador)
Administración -> Usuarios / Todos los hábitos / Estadísticas
```

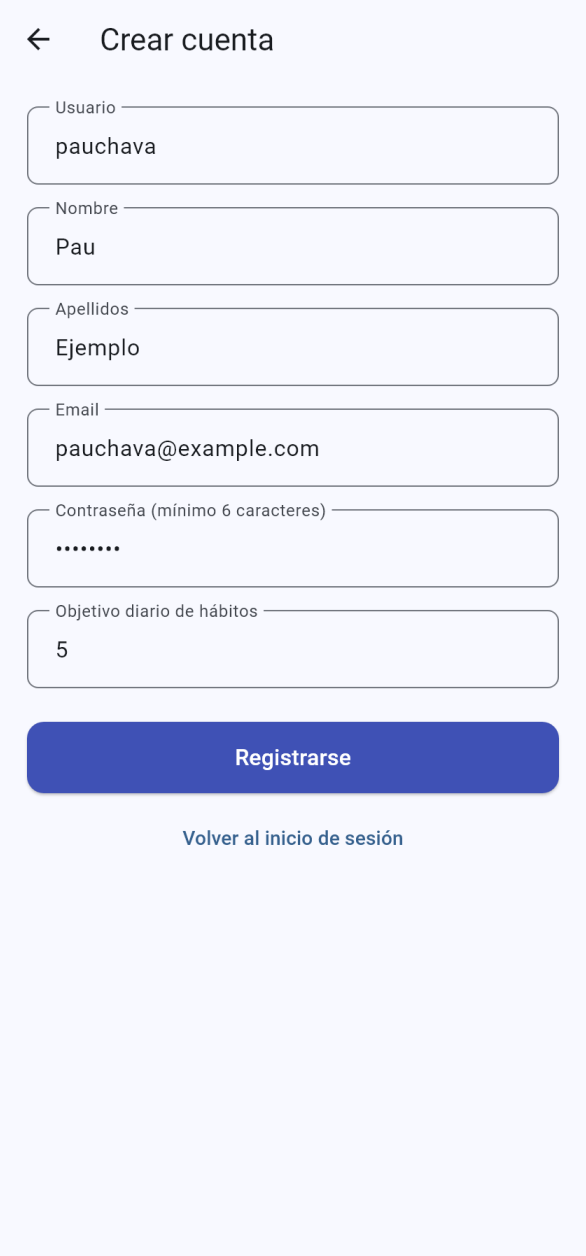
La pantalla de edición sirve tanto para crear como para modificar. Al guardar, se vuelve a la lista y se recargan los datos. Cancelar vuelve sin guardar. El borrado se confirma con un AlertDialog antes de llamar al servicio.

# 2. Inicio de sesión y registro

El acceso utiliza CampoTextoApp para usuario y contraseña, y BotonPrincipal para entrar. Crear cuenta abre el registro; desde este se puede volver al inicio de sesión.




Inicio de sesión: la contraseña se escribe oculta.

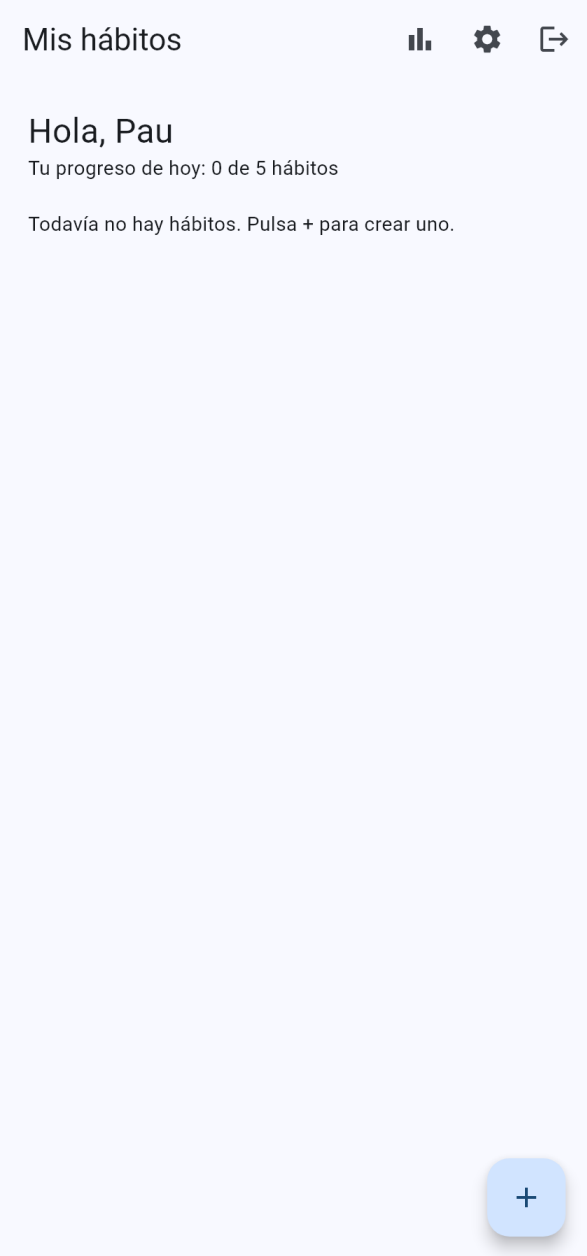



Registro con datos de ejemplo. La fecha de creación se asigna al guardar.

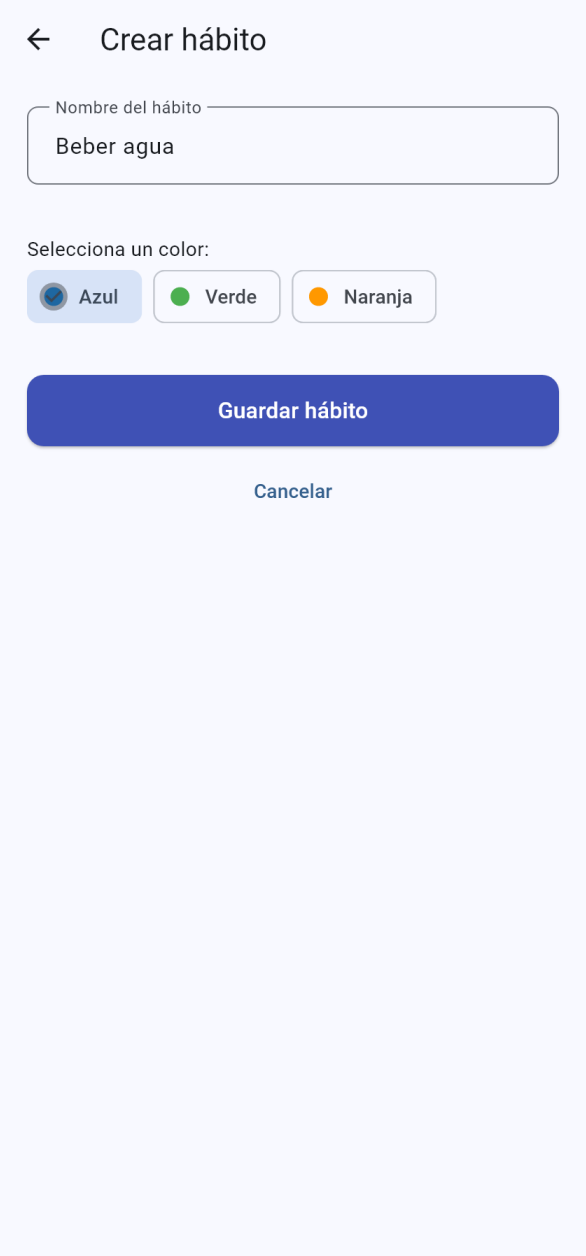
Se comprueba que los campos estén completos. El servicio verifica las credenciales y se muestra un mensaje si son incorrectas. La primera cuenta registrada es administradora; las siguientes reciben el rol de usuario.

# 3. Lista vacía y creación de un hábito

Tras entrar, la lista muestra los hábitos de la cuenta. Cuando no hay ninguno, aparece un mensaje y el botón + permite crear el primero.




Mis hábitos antes de crear el primer registro.

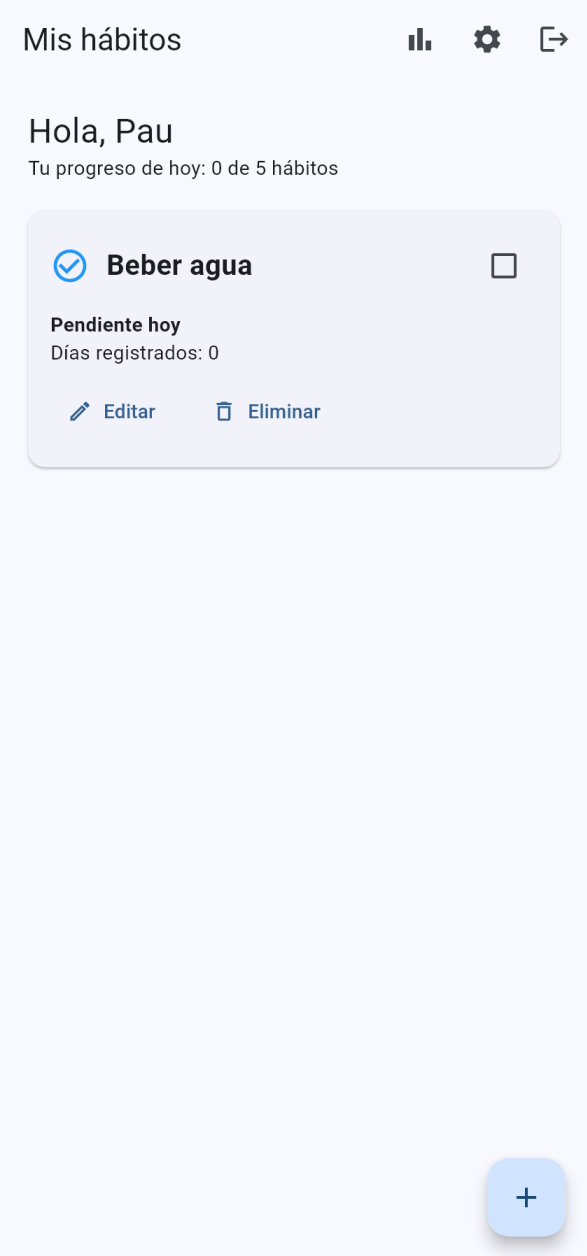



Creación de Beber agua, con nombre vacío al abrir y azul como color inicial.

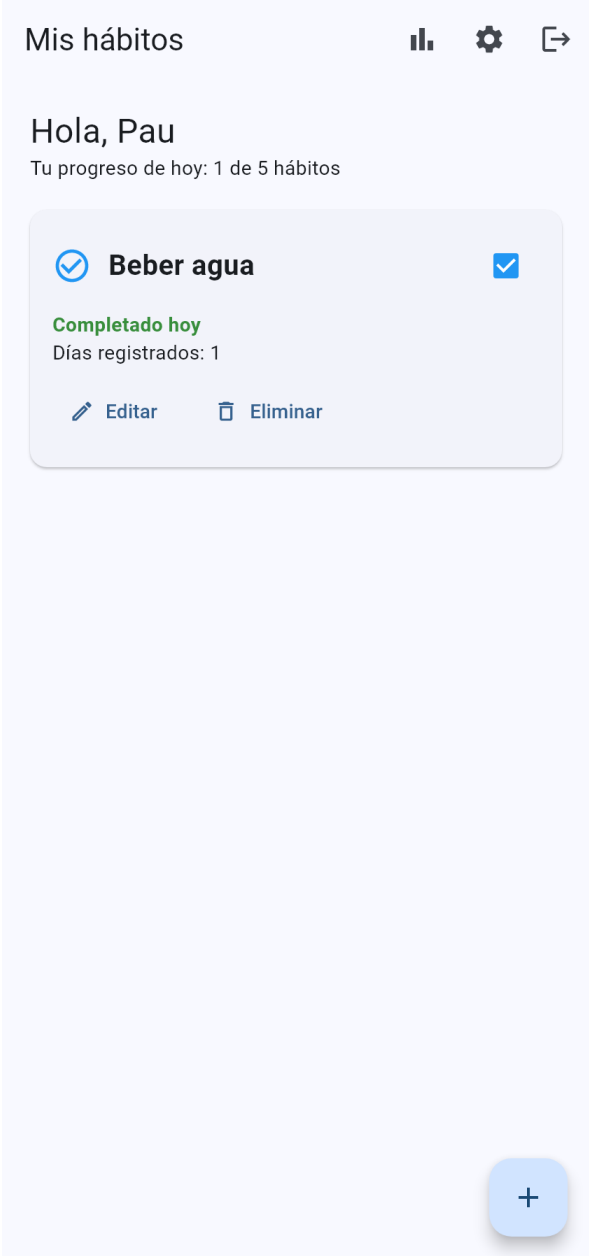
El nombre se introduce con CampoTextoApp. Los colores se eligen mediante botones predefinidos. Guardar hábito ejecuta la acción de la pantalla, guarda en SQLite y vuelve a la lista.

# 4. Crear y marcar como completado

El hábito recién creado aparece como pendiente. Su tarjeta comparte la misma estructura que las demás, pero recibe el nombre, el color y las acciones de ese registro.




Resultado de crear Beber agua: pendiente y progreso 0 de 5.

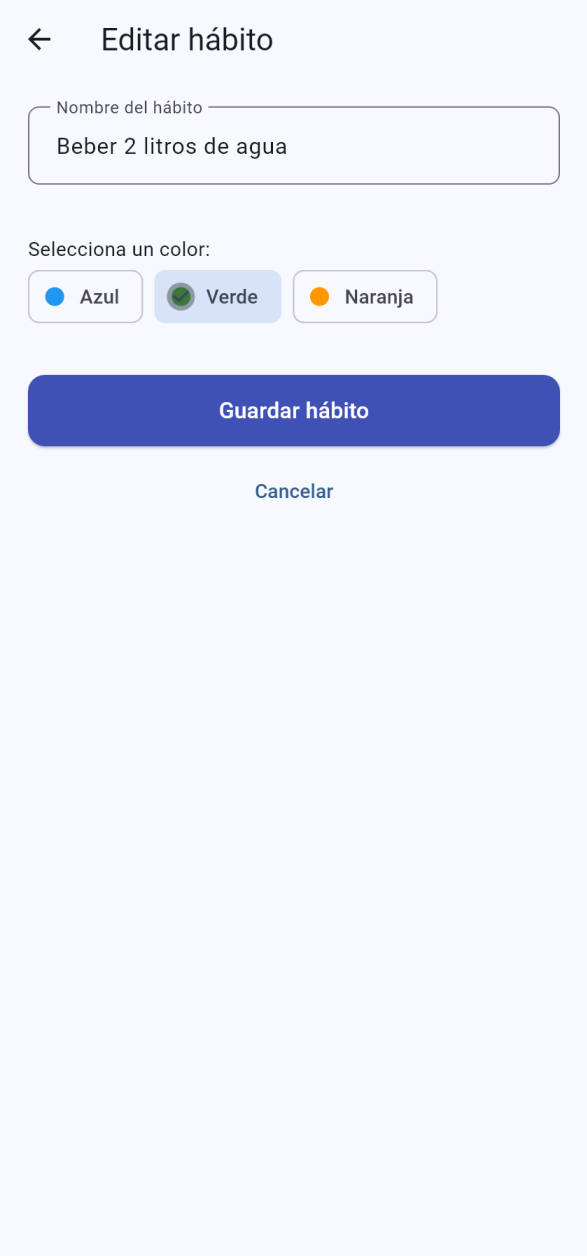



Al marcar la casilla: Completado hoy y progreso 1 de 5.

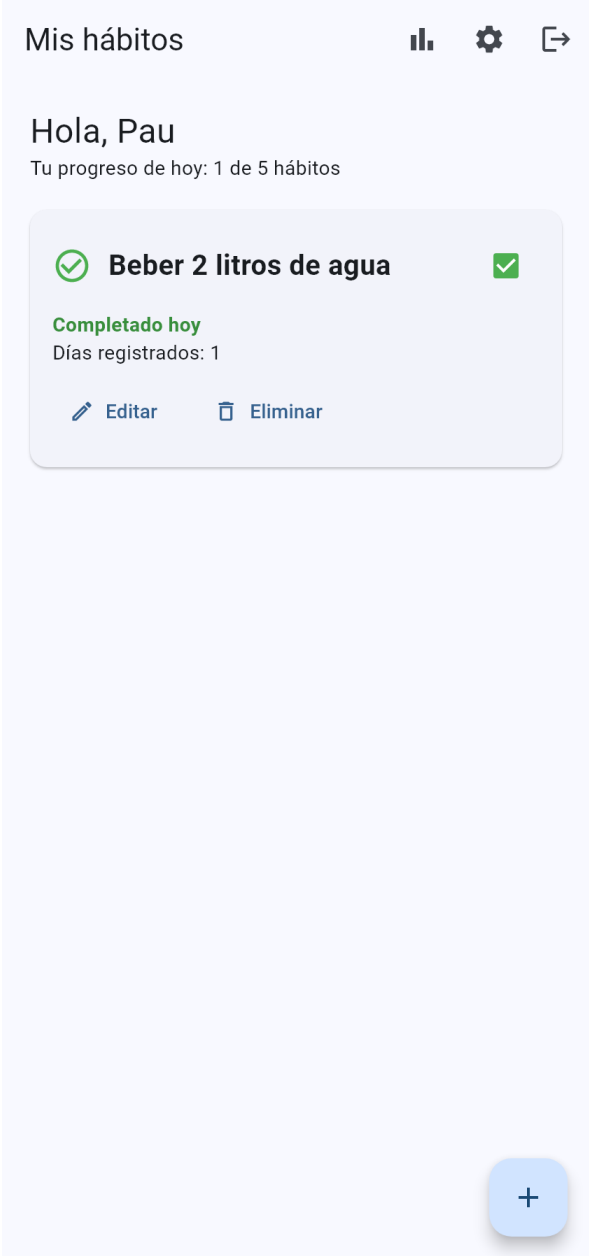
La casilla ejecuta alCompletar con un booleano. La pantalla llama al servicio para guardar la fecha de hoy y actualizar la estadística. Volver a desmarcar retira esa fecha; marcar dos veces no duplica el día.

# 5. Edición y actualización de la lista

Editar abre la misma pantalla de creación, con el nombre y el color actuales. En el ejemplo se cambia el nombre a Beber 2 litros de agua y el color a verde.




Formulario con el nuevo nombre y el color verde seleccionado.

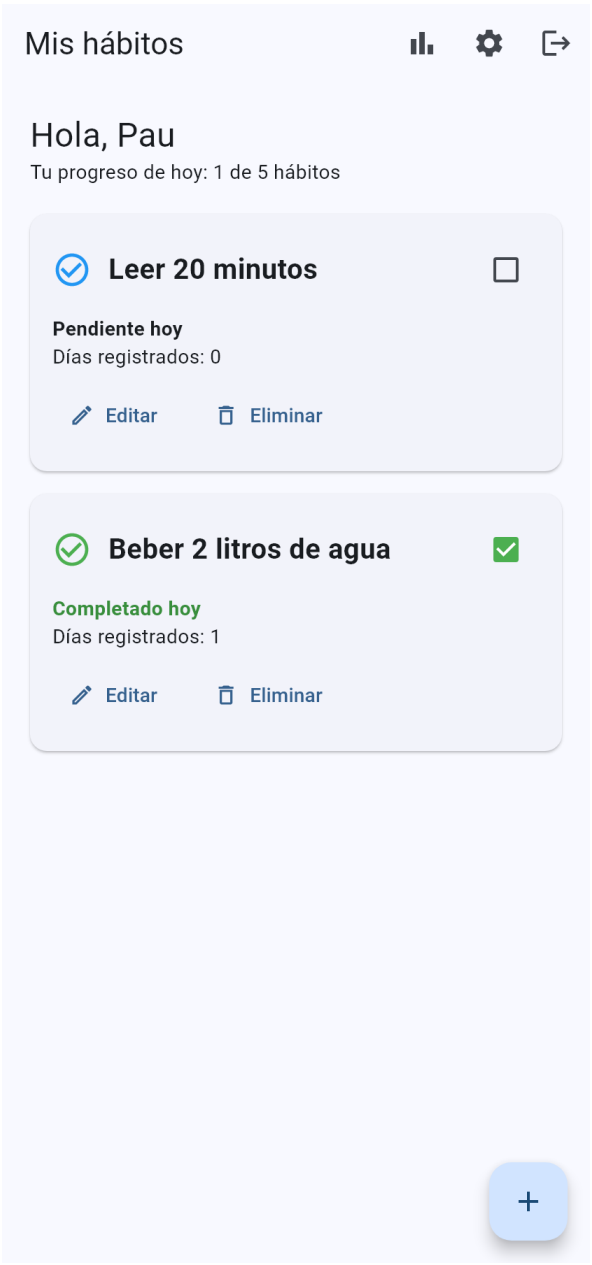



La lista refleja los cambios y conserva el día completado.

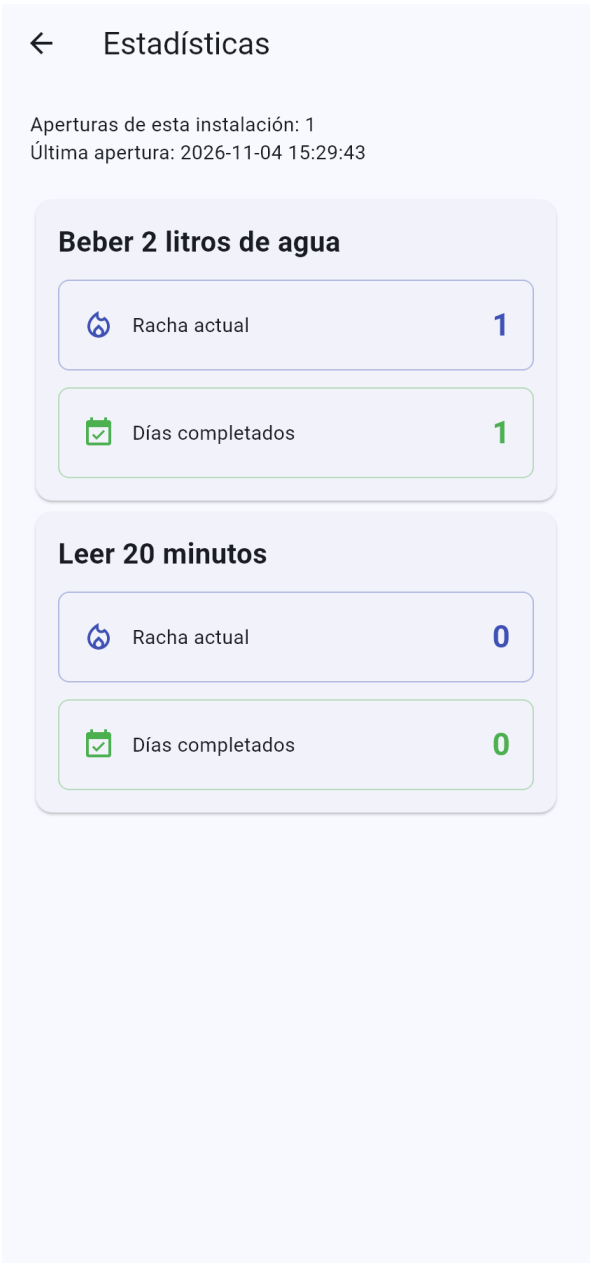
El callback alEditar está asociado al hábito de esa tarjeta. Guardar actualiza el registro existente, sin crear otro. Cancelar cierra el formulario sin modificar el hábito.

# 6. Reutilización y estadísticas del usuario

Se crea un segundo hábito, Leer 20 minutos. Las dos tarjetas utilizan TarjetaHabito y reciben datos distintos. Las estadísticas se agrupan bajo el nombre de cada hábito.




Dos instancias de TarjetaHabito: un hábito pendiente y otro completado.

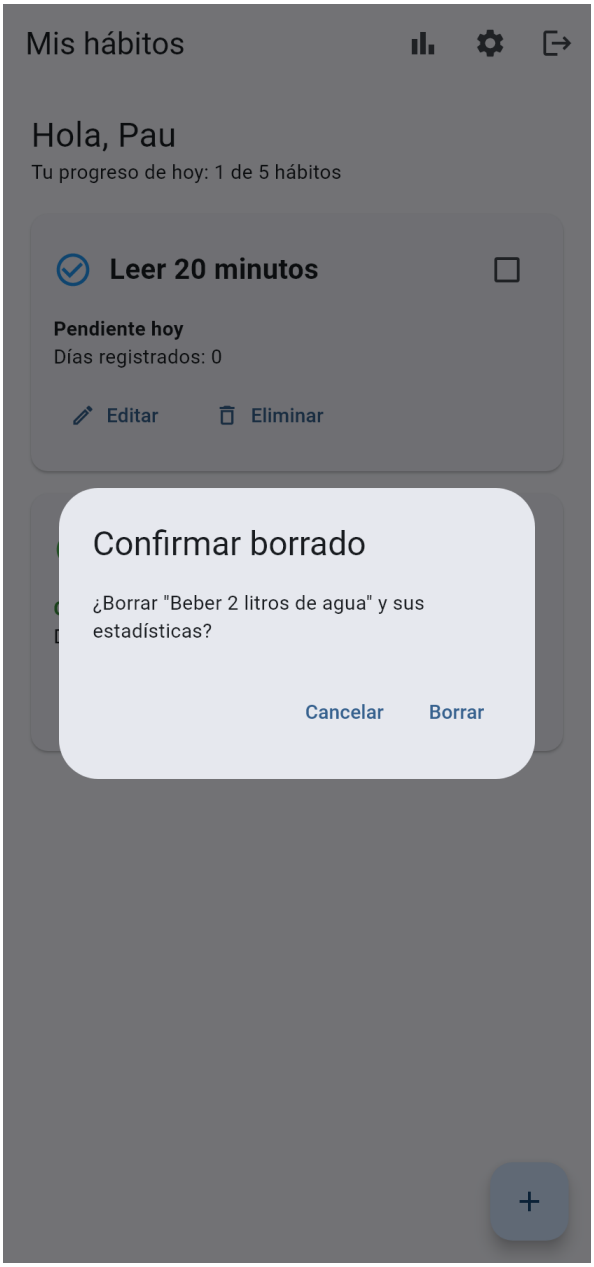



Cada hábito muestra dos instancias de DatoEstadistica.

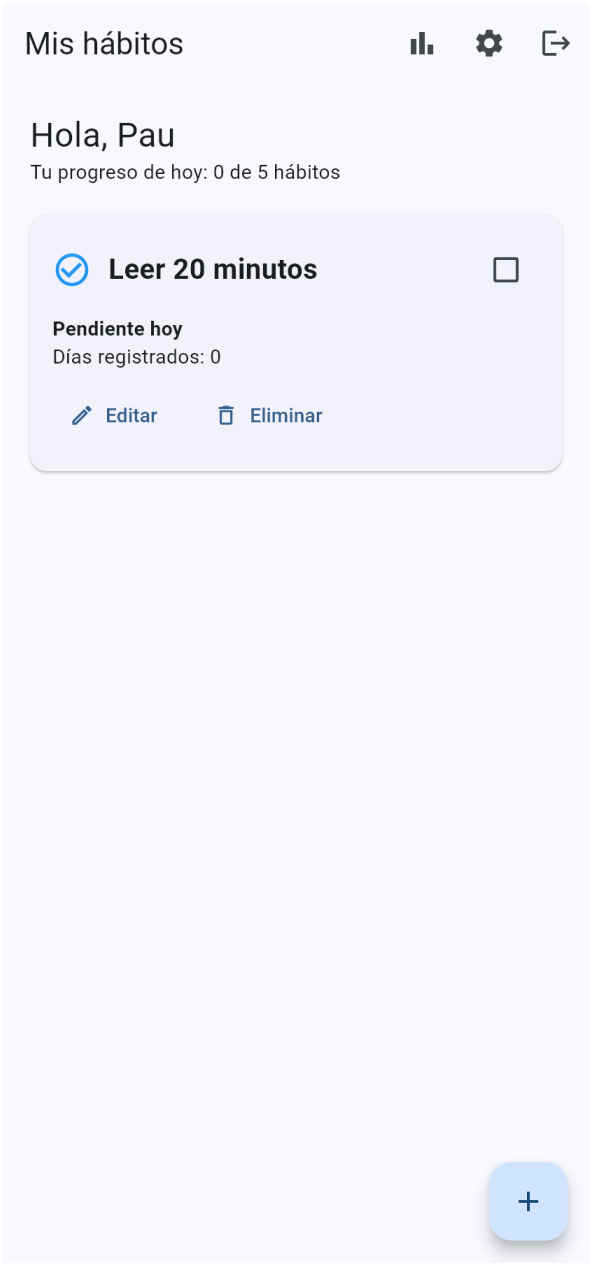
DatoEstadistica recibe una etiqueta, un valor y un icono. En esta pantalla sirve para la racha actual y para los días completados. La cuenta de usuario solo consulta sus propios hábitos y estadísticas.

# 7. Eliminación de un hábito

Eliminar en la tarjeta de Beber 2 litros de agua abre la confirmación. El componente solo comunica la acción; la pantalla decide cuándo pedir confirmación y llamar al servicio.




AlertDialog antes de borrar el hábito y su estadística.

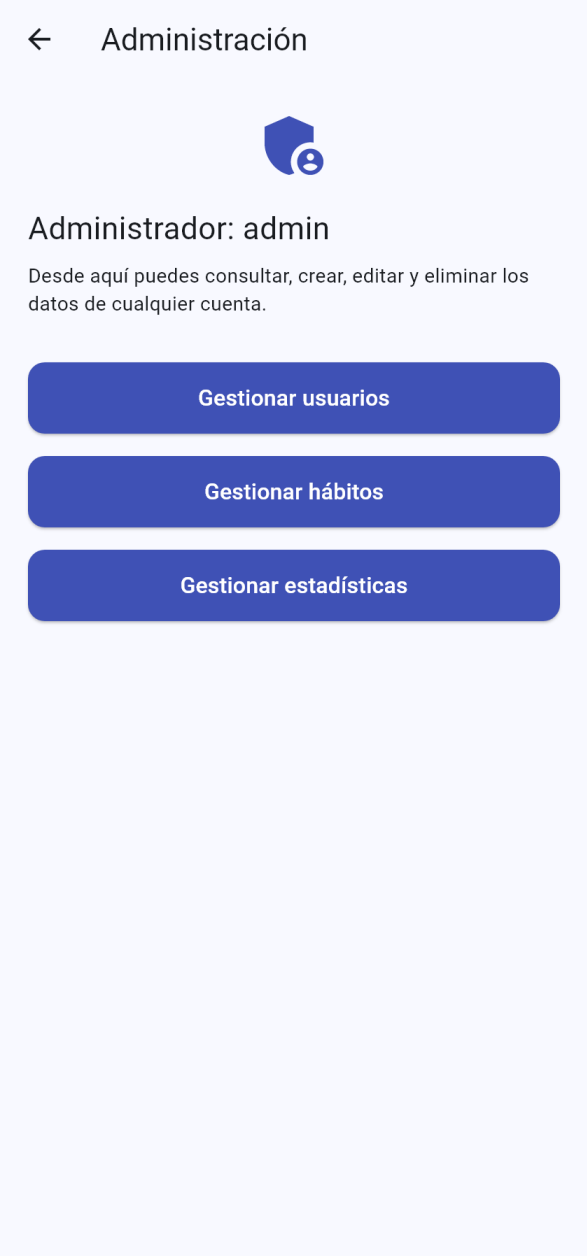



Después de confirmar, se conserva Leer 20 minutos.

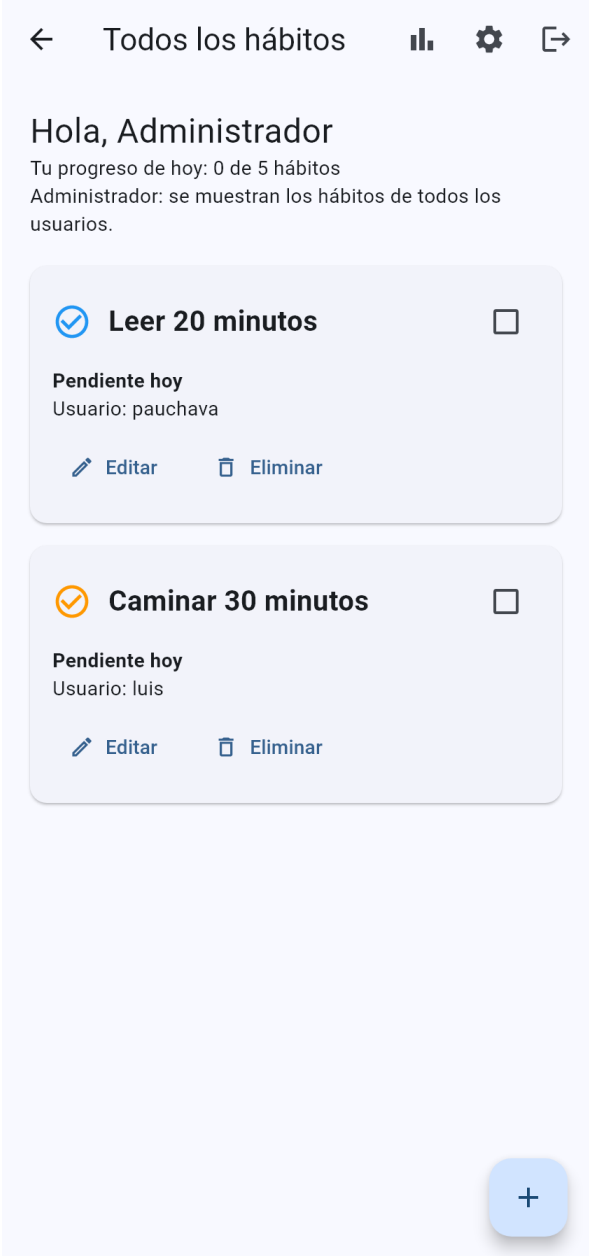
Cancelar en el diálogo conserva el registro. Borrar confirma la operación, elimina el hábito y su estadística, y recarga la lista. Cada tarjeta llama al callback del hábito correspondiente.

# 8. Administración y acceso a todos los hábitos

Solo la cuenta administradora dispone del acceso a Administración. Esta pantalla reúne la gestión de usuarios, hábitos y estadísticas.




Pantalla de administración con los tres accesos de gestión.

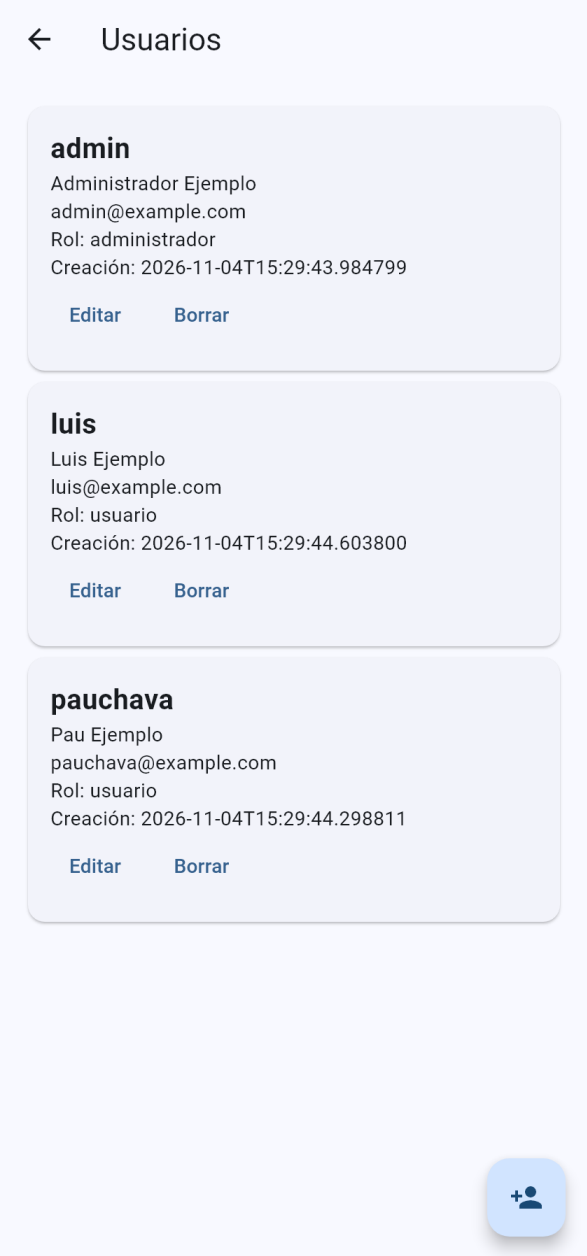



Todos los hábitos identifica a su propietario: pauchava o Luis.

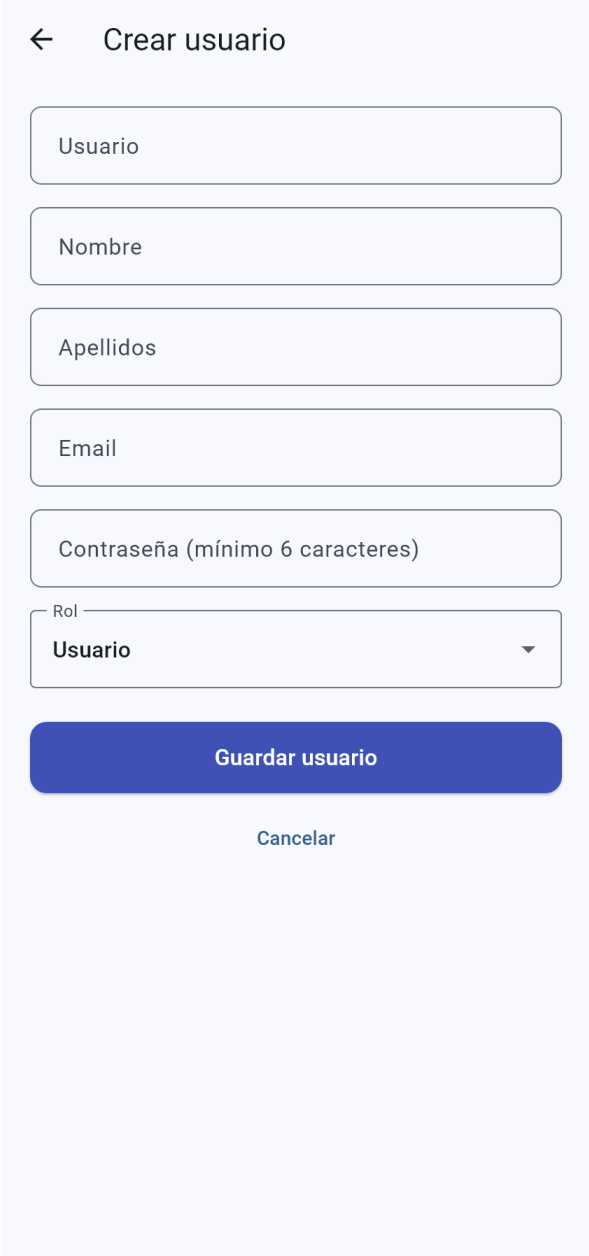
Mis hábitos muestra los registros personales, también para un administrador. Desde Administración se accede al conjunto de cuentas. El servicio comprueba el rol y la propiedad en las operaciones; ocultar el botón no es la única comprobación.

# 9. Gestión de usuarios

El administrador puede consultar usuarios, crear cuentas, editar sus datos y roles y eliminarlas. Se reutilizan CampoTextoApp y BotonPrincipal en el formulario de gestión.




Consulta de las cuentas de ejemplo y de sus roles.




Formulario para crear una cuenta desde administración.

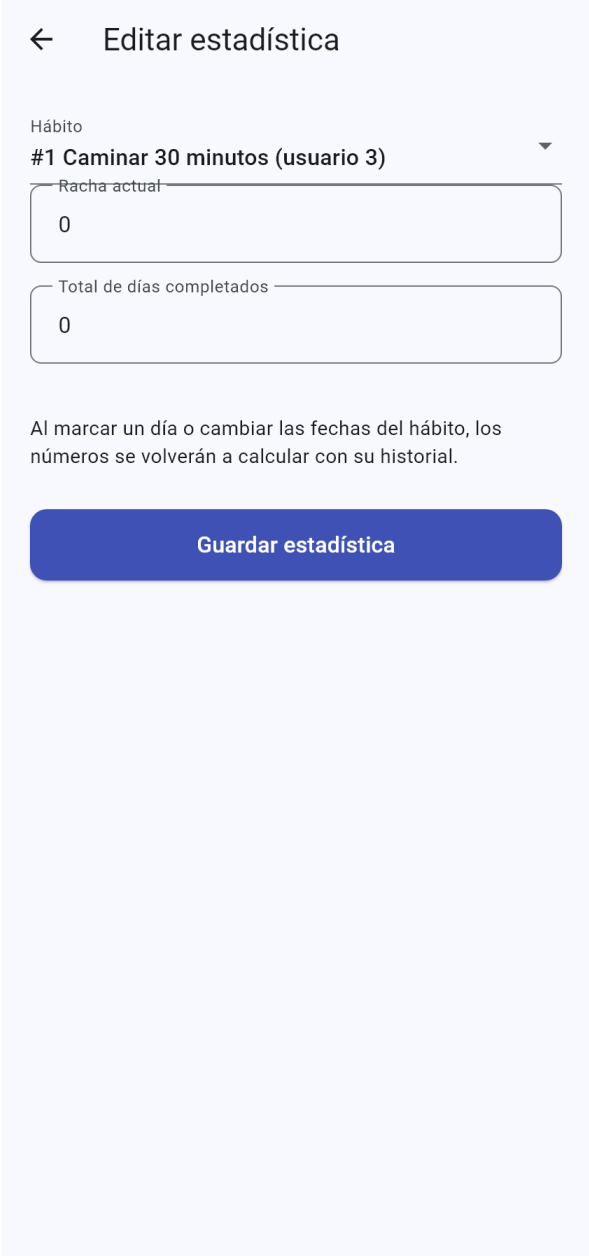
Al editar se cargan los datos existentes y una contraseña nueva es opcional. Debe quedar al menos un administrador. El borrado de una cuenta pide confirmación y elimina también sus hábitos y estadísticas.

# 10. Gestión de estadísticas

La vista de administración permite consultar las estadísticas de cualquier hábito y abrir su edición. Los datos visuales mantienen DatoEstadistica en las dos vistas.




Estadísticas con acciones de edición y borrado.

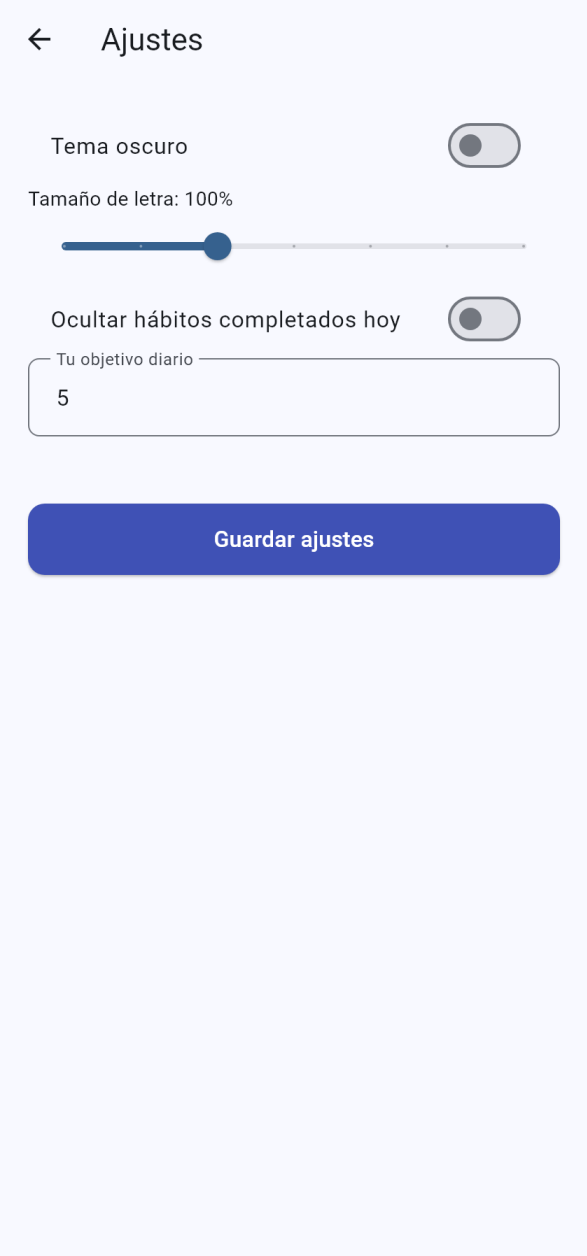



Formulario de una estadística, con hábito, racha y total.

El administrador puede crear una estadística cuando un hábito no la tiene, editarla o borrarla. La racha no puede superar el total. Al marcar un día o cambiar las fechas del hábito, los valores vuelven a calcularse con su historial.

# 11. Ajustes y organización de archivos

Los ajustes conservan el tema, el tamaño de letra, el objetivo diario y el filtro de hábitos completados. La organización de lib separa cada pantalla y cada componente en su archivo.




Ajustes de la cuenta, con el mismo campo y botón principal.

Organización del proyecto

```
lib/
    main.dart
    modelos/
        estadistica.dart
        habito.dart
        usuario.dart
    pantallas/
        administracion_page.dart
        ajustes_page.dart
        editar_habito_page.dart
        estadistica_form_page.dart
        estadisticas_page.dart
        habitos_page.dart
        login_page.dart
        registro_page.dart
        usuario_form_page.dart
        usuarios_page.dart
    servicios/
        base_datos_service.dart
        preferencias_service.dart
    widgets/
        boton_principal.dart
        campo_texto_app.dart
        dato_estadistica.dart
        dialogos.dart
        tarjeta_habito.dart
```

Captura del árbol real de carpetas y archivos del proyecto.

Las pantallas crean sus TextEditingController y los liberan en dispose. Los cuatro componentes obligatorios heredan de StatelessWidget. Los widgets no consultan SQLite ni SharedPreferences.

# 12. CampoTextoApp

**Parámetros:** etiqueta muestra el nombre del campo; controller lo crea la pantalla. ocultar vale false por defecto y se activa para contraseñas. tipoTeclado, maxLineas, pista y ayuda adaptan el campo. validador revisa el valor, alEnviar recibe el texto al confirmar y habilitado impide editar mientras se guarda.

```
import 'package:flutter/material.dart';
class CampoTextoApp extends StatelessWidget {
  final String etiqueta;
  final TextEditingController controller;
  final bool ocultar;
  final TextInputType tipoTeclado;
  final String? Function(String?)? validador;
  final ValueChanged<String>? alEnviar;
  final int maxLineas;
  final bool habilitado;
  final String? pista;
  final String? ayuda;
  const CampoTextoApp({
    super.key,
    required this.etiqueta,
    required this.controller,
    this.ocultar = false,
    this.tipoTeclado = TextInputType.text,
    this.validador,
    this.alEnviar,
    this.maxLineas = 1,
    this.habilitado = true,
    this.pista,
    this.ayuda,
  });
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 16),
      child: TextFormField(
        controller: controller,
        obscureText: ocultar,
        keyboardType: tipoTeclado,
        validator: validador,
        onFieldSubmitted: alEnviar,
        maxLines: ocultar ? 1 : maxLineas,
        enabled: habilitado,
        decoration: InputDecoration(
          labelText: etiqueta,
          hintText: pista,
          helperText: ayuda,
          border: OutlineInputBorder(borderRadius: BorderRadius.circular(8)),
          contentPadding: const EdgeInsets.all(16),
        ),
      ),
    );
  }
}
```

### Ejemplo de uso en login_page.dart

```
CampoTextoApp(
  etiqueta: 'Usuario',
  controller: usuarioController,
  validador: obligatorio,
  habilitado: !entrando,
)
```

Código completo del archivo del proyecto. Se han omitido las líneas en blanco.

# 13. BotonPrincipal

**Parámetros:** texto indica la acción; alPulsar es un VoidCallback, una función sin argumentos. habilitado vale true por defecto. Cada pantalla le pasa su propia acción: entrar, registrar o guardar. El tamaño de texto, el espaciado, el color y el borde se definen una sola vez.

```
import 'package:flutter/material.dart';
class BotonPrincipal extends StatelessWidget {
  final String texto;
  final VoidCallback alPulsar;
  final bool habilitado;
  const BotonPrincipal({
    super.key,
    required this.texto,
    required this.alPulsar,
    this.habilitado = true,
  });
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 8),
      child: SizedBox(
        width: double.infinity,
        child: ElevatedButton(
          onPressed: habilitado ? alPulsar : null,
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.indigo,
            foregroundColor: Colors.white,
            padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 14),
            textStyle: Theme.of(context).textTheme.labelLarge
                ?.copyWith(fontSize: 16),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
          child: Text(texto),
        ),
      ),
    );
  }
}
```

### Ejemplo de uso en login_page.dart

```
BotonPrincipal(
  texto: entrando ? 'Entrando...' : 'Entrar',
  alPulsar: entrar,
  habilitado: !entrando,
)
```

Código completo del archivo del proyecto. Se han omitido las líneas en blanco.

# 14. TarjetaHabito (1 de 2)

**Parámetros:** nombre, color y completadoHoy describen el hábito. icono tiene un valor por defecto. detalle añade información de la pantalla y habilitado controla las acciones. alCompletar recibe un bool; alEditar y alEliminar son VoidCallback.

```
import 'package:flutter/material.dart';
class TarjetaHabito extends StatelessWidget {
  final String nombre;
  final Color color;
  final bool completadoHoy;
  final IconData icono;
  final ValueChanged<bool> alCompletar;
  final VoidCallback alEditar;
  final VoidCallback alEliminar;
  final String? detalle;
  final bool habilitado;
  const TarjetaHabito({
    super.key,
    required this.nombre,
    required this.color,
    required this.completadoHoy,
    this.icono = Icons.check_circle_outline,
    required this.alCompletar,
    required this.alEditar,
    required this.alEliminar,
    this.detalle,
    this.habilitado = true,
  });
  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.only(bottom: 16),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Icon(icono, color: color, size: 28),
                const SizedBox(width: 12),
                Expanded(
                  child: Text(
                    nombre,
                    style: const TextStyle(
                      fontSize: 20,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),
```

El código continúa en la página siguiente.

# 14. TarjetaHabito (2 de 2)

```
                Checkbox(
                  value: completadoHoy,
                  activeColor: color,
                  onChanged: habilitado ? (valor) => alCompletar(valor!) : null,
                ),
              ],
            ),
            const SizedBox(height: 8),
            Text(
              completadoHoy ? 'Completado hoy' : 'Pendiente hoy',
              style: TextStyle(
                fontWeight: FontWeight.bold,
                color: completadoHoy ? Colors.green.shade700 : null,
              ),
            ),
            if (detalle != null) Text(detalle!),
            const SizedBox(height: 8),
            Wrap(
              spacing: 12,
              children: [
                TextButton.icon(
                  onPressed: habilitado ? alEditar : null,
                  icon: const Icon(Icons.edit_outlined),
                  label: const Text('Editar'),
                ),
                TextButton.icon(
                  onPressed: habilitado ? alEliminar : null,
                  icon: const Icon(Icons.delete_outline),
                  label: const Text('Eliminar'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

### Ejemplo de uso en habitos_page.dart

```
TarjetaHabito(
  nombre: habito.nombre,
  color: colorDelHabito(habito.color),
  completadoHoy: habito.completadoHoy,
  habilitado: !ocupado,
  alCompletar: (valor) => marcar(habito, valor),
  alEditar: () => editarHabito(habito),
  alEliminar: () => borrar(habito),
)
```

Las funciones conservan la referencia a habito en cada vuelta de la lista. Por eso editar o eliminar actúa sobre esa tarjeta. La confirmación del borrado, la llamada al servicio y setState permanecen en la pantalla.

# 15. DatoEstadistica

**Parámetros:** etiqueta indica qué dato se muestra; valor es un String; icono identifica el dato. color es opcional y utiliza índigo por defecto. El componente no calcula estadísticas: muestra los valores recibidos de la pantalla.

```
import 'package:flutter/material.dart';
class DatoEstadistica extends StatelessWidget {
  final String etiqueta;
  final String valor;
  final IconData icono;
  final Color color;
  const DatoEstadistica({
    super.key,
    required this.etiqueta,
    required this.valor,
    required this.icono,
    this.color = Colors.indigo,
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      margin: const EdgeInsets.only(top: 12),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        border: Border.all(color: color.withValues(alpha: 0.35)),
        borderRadius: BorderRadius.circular(8),
      ),
      child: Row(
        children: [
          Icon(icono, color: color),
          const SizedBox(width: 12),
          Expanded(child: Text(etiqueta)),
          const SizedBox(width: 12),
          Text(
            valor,
            style: TextStyle(
              fontSize: 22,
              fontWeight: FontWeight.bold,
              color: color,
            ),
          ),
        ],
      ),
    );
  }
}
```

### Ejemplos de uso en estadisticas_page.dart

```
DatoEstadistica(
  etiqueta: 'Racha actual',
  valor: '${estadistica.rachaActual}',
  icono: Icons.local_fire_department_outlined,
)
DatoEstadistica(
  etiqueta: 'Días completados',
  valor: '${estadistica.totalDias}',
  icono: Icons.event_available,
  color: Colors.green,
)
```

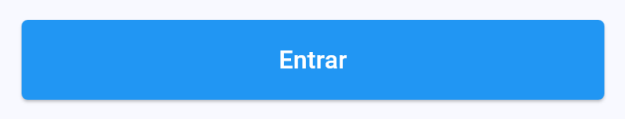
# 16. Un cambio visual en un solo componente

Primero se capturó BotonPrincipal con fondo azul y radio 4. Después se modificaron dos valores del mismo archivo: fondo índigo y radio 12. Las pantallas conservaron sus usos y sus callbacks.

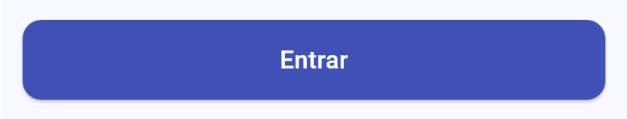
| Antes | Después |
| --- | --- |
| backgroundColor: Colors.blue | backgroundColor: Colors.indigo |
| BorderRadius.circular(4) | BorderRadius.circular(12) |

### Inicio de sesión: Entrar

Antes

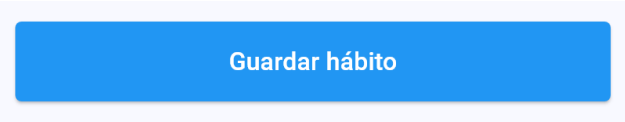



Después

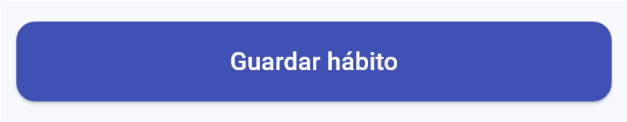



### Crear hábito: Guardar hábito

Antes




Después




Los detalles anteriores son zonas de las capturas completas de las dos pantallas. Las capturas iniciales y el código anterior se conservan junto a esta memoria.

### Callbacks y comprobación de la reutilización

Un callback es una función que se pasa por constructor. En el botón, alPulsar ejecuta la acción de esa pantalla. En la tarjeta, alCompletar comunica el nuevo valor de la casilla y alEliminar avisa a la pantalla para que pida confirmación. Los componentes no necesitan conocer la base de datos.

Las pruebas comprueban que CampoTextoApp oculta la contraseña, que BotonPrincipal respeta habilitado, que dos tarjetas ejecutan acciones distintas y que DatoEstadistica muestra etiquetas y valores diferentes. También se han comprobado las consultas, los permisos y el guardado de SQLite.

**Resultado:** 15 pruebas automáticas correctas y un recorrido adicional correcto de navegación y capturas. El análisis del código no detecta incidencias. La captura documental se ejecuta de forma explícita y se omite en una ejecución normal de las pruebas.